

Hermes Agent Skill Set 与 Skill 自进化机制 综合总结

基于两个上传文件的统一学习版 PDF

来源:

1. Hermes_Agent_Skill_Set_源码学习文档(1).docx
2. Hermes Skill 自进化机制解析.pdf

生成日期: 2026 年 7 月 5 日

目录

- 0. 执行摘要
- 1. 关键概念与边界
- 2. Skill Set 总体架构
- 3. Skill 的文件级表示
- 4. 索引、查询与渐进式披露
- 5. 自进化的三条闭环
- 6. Background Review 机制
- 7. skill_manage 写入动作
- 8. 安全机制与生命周期治理
- 9. 主仓库自进化 vs 独立 self-evolution
- 10. 工程启示与研究建议
- 11. 一页速记版

0. 执行摘要

一句话结论：Hermes 的 Skill 不是普通提示词，也不是向量库中的 JSON 条目，而是以目录为单位的“文件夹式程序性记忆”。运行时先把 Skill 的名称、短描述和类别压缩进系统提示，由 LLM 根据用户意图选择，再通过 skill_view 按需加载完整 SKILL.md 或支持文件；所谓自进化，本质是把成功流程、用户纠正、debug pattern 和可复用工具用法写回这些文件，而不是训练模型参数。

- Skill 表示：一个 Skill 通常是一个目录，入口是 SKILL.md，旁边可以有 references、templates、scripts、assets 等支持文件。
- Skill Index：主链路不是 embedding 检索、BM25 或 reranker，而是“文件系统扫描 -> Frontmatter 元数据 -> 紧凑目录 -> LLM 语义路由”。
- Skill 使用：skills_list 负责枚举摘要；skill_view 负责精确加载完整技能或某个支持文件；加载成功后会记录 view/use 遥测。
- 自进化：由任务中主动 skill_manage、任务后 Background Review、长期 Curator 治理三条闭环组成。
- 治理：.usage.json 记录使用情况；Curator 根据 activity timestamp 执行 active -> stale -> archived 迁移，并可选做 LLM consolidation。
- 安全边界：read-before-write、Background Write Guard、write_approval、人审 pending 队列、可选 guard_agent_created 安全扫描共同限制后台 Agent 乱改技能。
- 外部 self-evolution 仓库：DSPy + GEPA 的评测驱动优化流水线，属于离线候选搜索，不是主仓库每轮对话自动执行的参数学习。

问题	综合结论
Hermes Skill 到底是什么？	可复用程序性知识包：目录 + SKILL.md + 支持文件。
Skill Set 是数据库吗？	不是固定数据库对象，而是本地目录、外部目录、插件注册表动态解析得到的有效集合。
Skill Index 是向量索引吗？	不是。它是系统提示中的结构化文本目录，由 LLM 自行语义匹配。
自进化是不是强化学习？	主仓库不是强化学习或在线微调；它是文件级知识库修改。独立 self-evolution 更像评测反馈驱动的候选搜索。
最核心的闭环是什么？	经验发现 -> 写入程序性记忆 -> 下次按需加载 -> 记录使用 -> 生命周期治理。

1. 关键概念与边界

理解 Hermes Skill 必须先区分 Skill、Memory、Index、Evolution 和外部 self-evolution。它们分别解决不同层次的问题，混在一起会导致对“自进化”的误解。

概念	含义	常见误解
Skill	完成某类任务的可复用程序性知识包，描述“如何做”。	不是单条 prompt，也不是模型权重。
Memory	关于用户、环境或世界的声明性事实，描述“是什么”。	不要把用户事实写成操作技能。
Skill Set	当前运行环境可发现、可展示、可加载的 Skill 动态集合。	不是长期驻留的单一 SkillSet 数据库对象。

Skill Index	向 LLM 展示的名称 / description / category 紧凑目录。	不是 ANN 向量索引，也没有 Top-k 相似度分数。
Skill Evolution	通过 create、patch、edit、write_file、archive 等文件操作演进技能库。	不是梯度训练、RLHF 或在线微调。
Self-Evolution 仓库	用 DSPy + GEPA 读取 skill/prompt/tool/code，生成候选并通过评测选择。	不是 Hermes 主进程每个 turn 自动运行的机制。

边界判断：如果内容是“用户使用 Windows”，它更像 Memory；如果内容是“PowerShell 下如何正确定位 ~/.hermes 路径并避免 ~ 不展开的问题”，它更像 Skill。

2. Skill Set 总体架构

Hermes 没有把 Skill 固化为一个集中式数据库。有效 Skill Set 来自运行时解析：本地 ~/.hermes/skills/、配置中的 external_dirs，以及 namespace:skill-name 形式的插件 Skill。仓库随项目分发的 bundled skills 会同步到用户目录，并通过 .bundled_manifest 保存来源哈希，避免用户本地修改被静默覆盖。

Runtime Skill Set

- Local skills: ~/.hermes/skills/
- External dirs: config.yaml -> skills.external_dirs
- Plugin skills: namespace:skill-name

优先级：本地目录同名 Skill > 外部目录同名 Skill
所有权：用户本地 Skill 可管理；external / plugin / hub / protected built-ins 默认更保守

总体数据流可以抽象为一个“元数据常驻、正文按需”的渐进链路：

```
SKILL.md + support files
  ↓ scan / parse frontmatter
Compact metadata index: name + description + category
  ↓ injected into system prompt
LLM interprets user intent
  ↓ tool call
skill_view(name[, file_path])
  ↓ full instructions loaded
Tool execution / task completion
  ↓ telemetry + reflection
skill_manage / Background Review / Curator
```

3. Skill 的文件级表示

Skill 的基本单元是目录，SKILL.md 是入口。支持文件不会被扫描成独立技能，而是通过 skill_view(name, file_path=...) 在需要时加载，从而实现 progressive disclosure。

```
~/.hermes/skills/research/arxiv-search/
├── SKILL.md           # 必需：Frontmatter + 主流程
├── references/        # 长说明、协议、API、规则
```

```
|—— templates/          # 输出模板、报告模板
|—— scripts/            # 可执行脚本
|—— assets/             # JSON、示例数据、静态资源
```

SKILL.md 通常由两层构成：上方 YAML Frontmatter 用于发现、过滤、配置和路由；下方 Markdown 正文用于实际执行流程。

```
---
name: arxiv-search
description: Search, filter and summarize papers from arXiv
version: 1.0.0
platforms: [linux, macos, windows]
metadata:
  hermes:
    category: research
    tags: [paper, arxiv, survey]
    requires_toolsets: [web]
    related_skills: [paper-review]
---
# arXiv Search
## When to Use
## Procedure
## Pitfalls
## Verification
```

字段	运行时作用	设计要点
name	逻辑标识，用于索引、查找、去重和遥测 key。	稳定、短、能表达任务语义。
description	系统提示索引中的主要路由信号。	前 60 个字符必须有区分度，避免空泛口号。
platforms / environments	offer-time relevance gate 和兼容性过滤。	平台限制要谨慎，避免过度隐藏。
requires_* / fallback_for_*	根据工具可用性决定是否展示或作为降级能力。	用于硬依赖与替代路径。
metadata.hermes.config	声明技能配置项，加载时注入当前值。	非敏感配置可以放这里，密钥不应放 Frontmatter。

4. 索引、查询与渐进式披露

主运行链路的 Skill Index 本质上是结构化文本目录，不是向量索引。系统会扫描 SKILL.md Frontmatter，经过平台、环境、工具依赖、disabled list、fallback 条件过滤，再把 name、description、category 渲染成可读目录注入系统提示。

```
research:
- arxiv-search: Search/filter/summarize arXiv papers...
```

- paper-review: Critique methods, experiments and claims...
github:
- github-code-review: Review pull requests and code changes...

因此，“检索到某个 Skill”并不是向量库返回 Top-k，而是 LLM 在系统提示目录中看到它，并判断它与当前用户任务相关，然后主动调用 skill_view。

工具	输入	输出	职责边界
skills_list	可选 category	可用 Skill 的 name、description、category	枚举与浏览，不计算相似度分数。
skill_view	name，可选 file_path	完整 SKILL.md 或支持文件内容	精确加载、路径安全检查、配置预处理、返回 readiness 信息。

渐进式披露的三层上下文：Level 0 常驻短索引；Level 1 被选中时加载完整 SKILL.md；Level 2 只有需要时才读取 references、templates、scripts、assets 中的具体文件。

Context cost $\approx$ metadata_index
+ I(selected) $\times$ SKILL.md
+ I(needed) $\times$ support_file

缓存机制也服务于这一点：进程内 LRU 避免重复扫描，磁盘快照保存本地技能的索引元数据和 manifest；但完整正文仍在 skill_view 阶段读取原始文件。

5. 自进化的三条闭环

两个源文件共同指向同一个核心判断：Hermes 自进化不是“模型自己训练自己”，而是三条文件级闭环共同维护程序性记忆。

闭环	触发/过程	结果
闭环一：任务中使用 Skill	skills_list -> skill_view -> 加载 SKILL.md / 支持文件 -> 执行任务。	记录 view/use 遥测，为后续活跃度判断提供依据。
闭环二：任务后复盘 Skill	run_conversation 结束 -> TurnFinalizer 判断 -> spawn_background_review -> 后台 review agent 判断 create/patch/write_file。	把成功流程、用户纠正、非平凡修复沉淀为程序性知识。
闭环三：长期治理 Skill	skill_usage 记录活动 -> Curator 周期维护 -> stale/archive/consolidation。	清理低价值或长期未用技能，合并重复微技能。

经验发现
↓
写入程序性记忆：SKILL.md / references / templates / scripts / assets
↓
下次索引失效并重建，LLM 能看到更新后的能力目录
↓

任务中再次加载和使用
↓
usage telemetry 进入生命周期治理

6. Background Review 机制

Background Review 是主仓库在线自学习最核心的机制。它在用户当前任务响应交付之后运行，不继续回答用户，也不调用普通任务工具；它 fork 一个后台 review_agent，读取当前会话快照和工具轨迹，只开放 memory 与 skill management 相关工具，专门判断是否应该沉淀经验。

触发逻辑可以概括为：工具调用迭代累计到 _skill_nudge_interval，且 skill_manage 工具可用；本轮正常结束、有最终回复、没有被中断；TurnFinalizer 才会 spawn_background_review(review_skills=True)。

工具调用迭代增加 _iters_since_skill
↓
达到 _skill_nudge_interval
↓
本轮任务正常结束且未中断
↓
TurnFinalizer 触发 Background Review
↓
后台 review_agent 只允许 memory/skill 工具
↓
优先更新已加载 Skill，其次 umbrella Skill，最后才创建新 class-level Skill

应当学习的信号	不应保存的内容
用户纠正了风格、工作流或方法。	环境依赖的临时故障。
任务中出现非平凡修复、绕过方案、debug pattern、tool pattern。	对工具“坏了”的一次性负面结论。
已加载 Skill 过时、错误、缺步骤或缺支持材料。	transient errors 和一次性任务叙事。
可复用流程可以抽象成类别级技能。	把每一轮会话都机械创建成一个新 Skill。

关键思想：保存 fix / procedure / reusable workflow，而不是保存失败叙事本身。这也是 Hermes 避免 Skill 库碎片化的主要 prompt 约束。

7. skill_manage 写入动作

Background Review 不直接写文件，它只能调用 skill_manage。skill_manage 是 Agent-managed skill creation/editing 工具，负责校验、审批、写入、回滚、清缓存和记录 provenance。

Action	作用	典型场景
create	创建新 Skill 目录并写入完整 SKILL.md。	没有合适 Skill，需要沉淀新的类别级流程。
patch	用 old_string/new_string 做精确替换，可支持 fuzzy matching。	修正已有步骤、命令或注意事项；小改动首选。

edit	整体替换 SKILL.md。	结构性重写或大规模重构。
write_file	新增或覆盖 references/templates/scripts/assets。	把长材料、脚本、模板从主文档下沉。
remove_file	删除支持文件。	清理过期材料。
delete	删除或归档 Skill，可传 absorbed_into 表示合并去向。	废弃、合并或显式用户决策。

写入链路通常包含：validate name/frontmatter/size/path、路径安全检查、原子写回、可选 security scan、失败回滚、清除 skills system prompt cache、更新 usage telemetry、标记 agent-created provenance。

源码限制也体现了工程边界：SKILL.md 最大约 100k 字符，支持文件单文件最大约 1MiB，Skill name 必须符合 lowercase/dot/underscore/hyphen 等规则。

8. 安全机制与生命周期治理

Hermes 的自进化能力有明显的安全闸门。它允许 Agent 写回经验，但不允许后台 Agent 无约束地凭空改文件。

机制	解决的问题	核心规则
read-before-write	防止后台 review 凭会话印象乱改 Skill。	Background Review 想 patch/edit/write_file，必须先通过 skill_view 读过目标 SKILL.md 或目标支持文件。
Background Write Guard	保护所有权边界。	拒绝后台修改 pinned、external paths、protected built-ins、hub-installed、bundled 等高风险目标。
write_approval	引入人审门。	开启后 create/edit/patch/delete/write_file/remove_file 进入 pending queue，用户 diff/approve/reject 后才落盘。
guard_agent_created	机器安全扫描。	默认关闭；开启后扫描 Agent 创建内容，发现危险结果时阻断或回滚。
provenance	区分用户资产与后台自治资产。	后台自动创建的 Skill 标记为 agent-created；前台用户创建的 Skill 更偏 user-owned。

长期治理由 tools/skill_usage.py 与 agent/curator.py 承担。.usage.json 作为 sidecar 保存 view/use/patch 次数、last_activity、state、pinned、created_by 等信息，避免运行遥测污染 SKILL.md Frontmatter。

```
~/hermes/skills/.usage.json
{
  "arxiv-search": {
    "view_count": 12,
    "use_count": 9,
    "patch_count": 3,
    "last_used_at": "...",
```



```
"state": "active",
"pinned": false,
"created_by": "agent"
}
}
```

Curator 的确定性路径不需要 LLM：默认约 7 天检查一次；长期无活动的 active Skill 约 30 天后标 stale，更长期无活动约 90 天后移入 .archive/；pinned、protected、cron 引用等对象会跳过自动迁移。consolidation 默认关闭，只有配置启用或显式运行时才让 LLM 合并重复微技能。

```
active
| no activity >= stale_after_days
▼
stale
| no activity >= archive_after_days
▼
archived -> moved to ~/.hermes/skills/.archive/ -> restorable
```

9. 主仓库自进化 vs 独立 self-evolution

“别人说自进化还包括强化学习”通常来自把两层机制混在一起：主仓库的在线闭环是文件级程序性记忆修改；独立 hermes-agent-self-evolution 仓库是评测驱动的候选演化流水线，使用 DSPy + GEPA 产生候选、跑评测、过约束门控、再通过 PR 合入。它可能带有“反馈优化/候选选择”的味道，但不是主进程每轮对话对模型参数做 RL。

维度	主仓库在线自进化	独立 self-evolution
触发	任务后复盘或周期维护。	显式运行优化流水线。
依据	会话经验、工具轨迹、已加载 Skill、使用遥测、LLM 判断。	Eval dataset、execution trace、constraint gates。
修改对象	SKILL.md、references、templates、scripts、assets、usage sidecar、archive 状态。	skills、tool descriptions、system prompts、code。
候选竞争	通常直接 create/patch/write_file，竞争较弱。	多候选变体生成、评测、比较与选择。
风险控制	read-before-write、write approval、background guard、scanner、Curator。	tests、size limit、semantic preservation、PR review 等门控。
本质	在线经验写回与生命周期治理。	离线评测驱动的演化搜索。

10. 工程启示与研究建议

从 Hermes 的设计可以抽出几条对 Agent Skill 系统很有价值的工程原则。

- Skill 应该是“类别级过程”，不是“一次性任务记录”。否则 Skill 库会快速膨胀，路由信号被噪声稀释。
- Description 是路由入口，前 60 个字符必须包含任务、触发条件和边界。不要把最关键的信息放到后半段。

- 主文档保持短而可执行；长规则、模板、脚本下沉到 references/templates/scripts，依赖 skill_view(file_path) 二级加载。
- 在线学习必须有 read-before-write 与 ownership guard，否则后台 Agent 会把偶然经验写成长期规则。
- usage telemetry 不应污染可共享的 SKILL.md，适合放 sidecar，并以原子替换和文件锁降低并发风险。
- 生命周期治理要区分 active/stale/archived，而不是直接删除；archive 可恢复，风险更低。
- 要进一步研究，可以补足 need awareness 与 relevance awareness：模型何时知道需要 Skill、何时知道当前 Skill 不够、何时组合多个 Skill。
- 如果要做更强自进化，应引入 eval-driven loop：候选生成、自动评测、约束门控、回归测试和人工 review，而不是只依赖会话后的主观 patch。

对研究方向而言，Hermes 代表的是“文件系统程序性记忆 + LLM 路由 + 使用遥测治理”的路线。它比纯提示词更工程化，比向量条目更能携带脚本和模板，但也依赖 LLM 对短描述的路由能力；当技能数量很大时，可能需要更强的分层目录、语义索引或训练过的 skill router。

11. 一页速记版

可以按下面这套话术理解 Hermes Skill：

- Hermes Skill 是程序性记忆，保存“怎么做某类任务”，不是保存用户事实。
- 一个 Skill 是一个目录，入口是 SKILL.md，支持 references、templates、scripts、assets。
- 运行时先扫描 Frontmatter，生成 name/description/category 的紧凑目录，放入 system prompt。
- LLM 看到目录后自己做语义路由；没有向量 Top-k、BM25 或 reranker。
- skill_view 是真正加载技能的动作：可加载完整 SKILL.md，也可加载支持文件。
- 自进化是文件级闭环：任务经验 -> Background Review -> skill_manage -> patch/create/write_file -> 缓存失效重建。
- 长期治理靠 usage sidecar 与 Curator：active -> stale -> archived，可选 consolidation。
- 安全靠 read-before-write、write_approval、Background Write Guard、agent-created scanner 和 provenance。
- 独立 self-evolution 是 DSPy + GEPA 的离线评测优化，不等于主仓库对模型做强化学习。

最终判断：Hermes 的“Agent grows with you”主要是外部 Skill 库随任务经验演进；模型权重本身不变，变的是可发现、可加载、可执行、可治理的程序性知识资产。